PARTICIPANT HANDOUT

Sharding Across Azure Storage Accounts

When to shard, how to know, how it works, how the logical maps to the physical, and the techniques available

Azure Storage, Data Protection, and Resilience at Scale

University of Minnesota custom course

Session 1, Block 3 supplement: scale targets and sharding

Companion to Reference Card 2 and the UMN Research Storage Inventory workbook

How to use this handout

Read sections 1 through 4 before the Block 3 discussion; they explain what a storage account is as a unit of quota and policy, and when a second account is forced rather than chosen.

Sections 5 through 7 explain the mechanics and the physical layer. Section 8 is the decision flowchart. Section 9 is a worked example built on the conventions of the Research Storage Inventory, which you will reuse in the Block 4 design exercise and again in the Session 6 capstone.

The knowledge check in section 11 has an answer key at the end. The numeric targets in this handout were verified against Microsoft Learn in October 2026; the scalability targets page is the authority if a number has moved.

Version 1.0, October 2026. Prepared by Gamut Technology Services. All dataset figures in the worked example are synthetic and modeled on the course inventory; no real University of Minnesota data appears in this document.

Contents

Contents 2
1. The problem sharding solves 4
1.1 The per-account targets 4
2. What an account decides once, for everything inside it 5
3. When to shard: three triggers, in priority order 5
3.1 Trigger 1: policy conflict (forced) 5
3.2 Trigger 2: blast radius and administration (forced) 6
3.3 Trigger 3: scale (a judgment call) 6
4. How you know: four signals 6
4.1 Reading the signals together 7
5. The mechanics: a sharding scheme is three things 7
5.1 A key 7
5.2 A map 7
5.3 A set of consequences 8
6. How the logical maps to the physical 8
6.1 The storage stamp 8
6.2 Three layers between a request and a disk 9
6.3 Three consequences for sharding 9
7. Naming is sharding one level down 10
8. The decision flowchart 11
9. Worked example from the Research Storage Inventory 12
9.1 The naming convention 12
9.2 Nine representative datasets 12
9.3 Walking each dataset through the flowchart 13
9.4 The resulting account set 14
9.5 The capacity projection, as the inventory computes it 14
10. The techniques, as a menu, and two clarifications 15
10.1 Containers are not shards 15
10.2 Geo-redundancy is not sharding 15
11. Knowledge check 15
12. Glossary 16
13. Microsoft Learn references 17
Answer key 18

If the field above is empty, right-click it and choose Update Field. The static list below carries the page numbers as rendered for print.

1. The problem sharding solves 3
2. What an account decides once, for everything inside it 4
3. When to shard: three triggers, in priority order 4
4. How you know: four signals 5
5. The mechanics: a sharding scheme is three things 6
6. How the logical maps to the physical 7
7. Naming is sharding one level down 8
8. The decision flowchart 10
9. Worked example from the Research Storage Inventory 11
10. The techniques, as a menu, and two clarifications 14
11. Knowledge check 15
12. Glossary 15
13. Microsoft Learn references 16
Answer key 17

1. The problem sharding solves

A storage account is the unit of quota in Azure Storage. Every target that matters to a petabyte portfolio is enforced per account: how much it can hold, how many requests per second it can serve, and how much data can flow in and out. Those limits are enforced in software by the platform, and they apply to the account as a whole regardless of how many containers or blobs are inside it.

Sharding is the practice of splitting a portfolio across more than one storage account so that no single account's quota, policy set, or blast radius becomes the constraint. On an on-premises frame you added shelves, controllers, and aggregates; in Azure you add accounts. That is the whole idea. Everything else in this handout is about deciding where the cut lines go, and understanding what a cut line does and does not buy you.

1.1 The per-account targets

The table below carries the default targets for standard general-purpose v2 accounts as published on Microsoft Learn in October 2026. The major-region figures apply in the regions Microsoft lists on the scalability targets page, which include Central US, East US, East US 2, North Central US, South Central US, West US, West US 2, and West US 3 among others. Capacity, ingress, and egress limits can be raised by request through Azure Support.

Resource	Default target	Notes
Storage account capacity	5 PiB	Higher by request
Request rate, major regions	40,000 requests per second	Combined across all operations on the account
Request rate, other regions	20,000 requests per second	
Ingress, major regions	60 Gbps	All data sent to the account
Ingress, other regions	25 Gbps	
Egress, major regions	200 Gbps	All data received from the account
Egress, other regions	50 Gbps	
Storage accounts per region per subscription	250 by default, 1,000 by request	Includes standard and premium accounts
Containers, blobs, shares, queues, tables per account	No limit	The account total, not the count, is what is capped
Single block blob request rate	Up to 3,000 requests per second	A per-object target; sharding across accounts does not change it
Single block blob throughput	Up to the account ingress and egress limits	Depends on concurrency, request size, and the far end
Maximum block blob size	About 190.7 TiB	50,000 blocks of up to 4,000 MiB

These are high-end targets, not guarantees. Microsoft's own wording is that they are achievable, and that the request rate and bandwidth an account reaches depend on object size, access pattern, and workload type. A workload of millions of small objects behaves very differently from a workload of large satellite granules, even at the same total capacity.

2. What an account decides once, for everything inside it

Before the triggers for sharding make sense, it helps to see how much is fixed at the account level. Several settings are chosen once per account and apply to every container and blob within it. Some of them cannot be changed after data lands; the rest can be changed but only for the account as a whole.

Decision	Scope	Reversible?	Why it matters for sharding
Account kind and performance tier (standard GPv2, premium block blob, premium file share)	Whole account	No	A workload that needs premium latency cannot share an account with standard-tier data
Hierarchical namespace (Data Lake Storage Gen2)	Whole account	One-way upgrade only	Hierarchical namespace accounts give up blob versioning, change feed, object replication, point-in-time restore, and version-level WORM
Redundancy (LRS, ZRS, GRS, GZRS, and the read-access variants)	Whole account	Conversion possible, with constraints	Archive tier is supported only on LRS, GRS, and RA-GRS accounts; a dataset that must be zone redundant and a dataset that must be archived cannot live together
Account name	Whole account	No	The name is the DNS entry and the first part of every partition key; choose the convention before the first account exists
NFS 3.0 or SFTP enabled	Whole account	Set at creation for NFS	Rules out immutability policies on that account
Shared key access, minimum TLS, public network access, firewall rules, private endpoints	Whole account	Yes	A locked-down archive and a collaboration share with external access cannot share one network posture
Keys, account-level RBAC, lifecycle policy, cost records, deletion	Whole account	Yes	This is the blast radius: whatever a key or an identity can reach, it reaches across the whole account

The point to hold onto

An account is not a folder. It is closer to a volume with a QoS policy, a single redundancy setting, a single namespace model, and its own keys. When two datasets need different answers to any of the decisions above, the second account is not a scaling choice; it is forced.

3. When to shard: three triggers, in priority order

Most people arrive at sharding expecting it to be about size. In practice, the first two triggers below force far more account boundaries than the third, and they should be checked first.

3.1 Trigger 1: policy conflict (forced)

If two datasets need different account-level settings, they cannot share an account. The common cases in a research portfolio:

- A polar satellite archive that will sit in the Archive tier (which requires LRS, GRS, or RA-GRS) and an active humanities corpus that the team wants zone redundant (ZRS) are in different accounts by necessity.
- A genomics pipeline that needs a hierarchical namespace for directory operations and an archive that needs version-level WORM are in different accounts, because hierarchical namespace accounts do not support version-level WORM, versioning, or object replication.
- A share exposed over NFS 3.0 and an immutable backup target are in different accounts, because enabling NFS 3.0 or SFTP rules out immutability policies on that account.
- A premium block blob account for low-latency scratch and a standard account for everything else; performance tier is fixed at creation.

This is the most common reason to shard and the one most often missed, because none of these cases has anything to do with capacity.

3.2 Trigger 2: blast radius and administration (forced)

An account is the scope of its keys, its account-level RBAC assignments, its private endpoints, its lifecycle policy, its cost records, and its deletion. That scope is the blast radius: a compromised key, a mis-scoped role, or a mistaken delete reaches everything in one account and nothing outside it. Several consequences follow.

- An immutable backup target never shares an account with the data it protects, and usually sits in a separate subscription owned by a different team. This is the same separation of duties the Session 3 Resource Guard design relies on.
- A research unit that bills separately gets cleaner showback from its own accounts. Tags can carry the allocation, but an account boundary makes the cost record unambiguous.
- A collaboration dataset that external partners reach over a different network posture does not share an account with restricted data.

3.3 Trigger 3: scale (a judgment call)

Measured or projected demand approaches an account target. Capacity is the easy case: the inventory gives you the growth rate and the arithmetic is straightforward. Request rate and bandwidth are harder, because they depend on access pattern rather than size. A 50 TB dataset of millions of 300 KB text files can exhaust a request-rate target that a 2 PB dataset of 20 MB granules never touches. This is why the inventory carries file count and average object size alongside capacity.

4. How you know: four signals

Every account exposes its own metrics in Azure Monitor. Four signals, read together, tell you whether a boundary is needed and which kind.

Signal	Where to read it	What to compare it to	What it tells you
Used capacity	Azure Monitor, UsedCapacity metric; the inventory growth rate for the projection	5 PiB default, or the raised limit on file with Azure Support	Whether and when a capacity split is needed. Plan it when the projection crosses roughly 60 to 70 percent of the target (a practitioner rule, not a Microsoft figure), because moving petabytes later is slow and billable

Signal	Where to read it	What to compare it to	What it tells you
Transactions, Ingress, Egress	Azure Monitor, account-level metrics with the API name and authentication dimensions	The per-region request rate, ingress, and egress targets in section 1.1	Whether the request profile or bandwidth of the workload is approaching the account's limit, independent of capacity
Throttling	Azure Monitor, Transactions metric split by the ResponseType dimension; look for ServerBusyError and ServerTimeoutError	Any sustained non-zero count	A hot partition or an account at its limit. This fires at the partition level, so an account well under its totals can throttle on one hot name range. The client should already be retrying with exponential backoff
Policy and ownership questions	The inventory columns: redundancy preference, immutability required, sensitivity, research unit, retention class	The account-level decisions in section 2	Whether a boundary is forced regardless of what the metrics say

4.1 Reading the signals together

- **Any policy or ownership question answered yes:** shard. The metrics are irrelevant.
- **Throttling appears:** fix naming first (section 7), then isolate the noisy workload in its own request-profile account.
- **Capacity, request rate, ingress, or egress trends toward a target:** shard the domain by key range before you reach it, and encode the shard sequence in the naming convention so the second account is a predictable name.
- **None of the above:** keep the data together. Fewer accounts are easier to govern, and the Session 6 Bicep module makes the accounts you do need cheap.

5. The mechanics: a sharding scheme is three things

5.1 A key

The key is the attribute that decides which account a dataset lands in. The course uses three, and the inventory carries all of them:

- **Data domain:** the research unit or project. This is the cleanest ownership boundary and the natural showback line.
- **Protection class:** the combination of redundancy, namespace model, and immutability requirement. This is the policy-conflict trigger expressed as a key.
- **Request profile:** active, cold, or archive, and whether the workload is transaction-heavy or bulk. This is the throttling trigger expressed as a key.

5.2 A map

The map is whatever answers the question "which account holds X." There are two forms, and a mature portfolio has both:

- **A deterministic naming convention.** The account name is a function of the key, so anyone who knows the research unit and data class can derive the account without looking it up. The worked example in section 9 uses `st` + `umn` + unit code + class code + two-digit sequence.
- **A catalog.** The inventory workbook, a table, or a metadata service that records the dataset-to-account mapping explicitly. A catalog survives renaming and rebalancing; a naming convention alone does not.

Having neither is the failure mode: a portfolio of accounts nobody can navigate without tribal knowledge.

5.3 A set of consequences

Nothing is atomic across accounts, and that has operational weight:

- There is no cross-account listing, no cross-account lifecycle rule, and no cross-account immutability policy. Each account carries its own.
- Each account has its own keys, RBAC assignments, private endpoints, diagnostic settings, and cost records. Ten accounts means ten of each, which is why they are created from one Bicep module with parameters rather than by hand.
- Moving data between accounts is a copy, not a move: AzCopy, object replication (flat namespace accounts only), or Azure Data Factory. The copy is billed as transactions, and across regions as egress.
- Tags (research unit, project, data class, retention class, owner) are what let the cost view add back up across accounts in Cost Management. Without them, sharding fragments the FinOps picture.

6. How the logical maps to the physical

The question that prompted this handout: when data is sharded across accounts, do the files in one shard sit on the same physical hardware? The answer is no, and understanding why reframes everything above.

6.1 The storage stamp

The physical unit of Azure Storage is the storage stamp: a cluster of racks of storage nodes inside one datacenter, hosting many storage accounts from many customers. When you create an account, the platform assigns it to a stamp in your region based on capacity and load (or to stamps in three availability zones for ZRS), and the account name becomes a DNS entry that routes to that stamp's front end. From that moment the account is a logical and administrative boundary only. It is the closest thing to a tenant with a quota; the stamp is the closest thing to a frame.

6.2 Three layers between a request and a disk

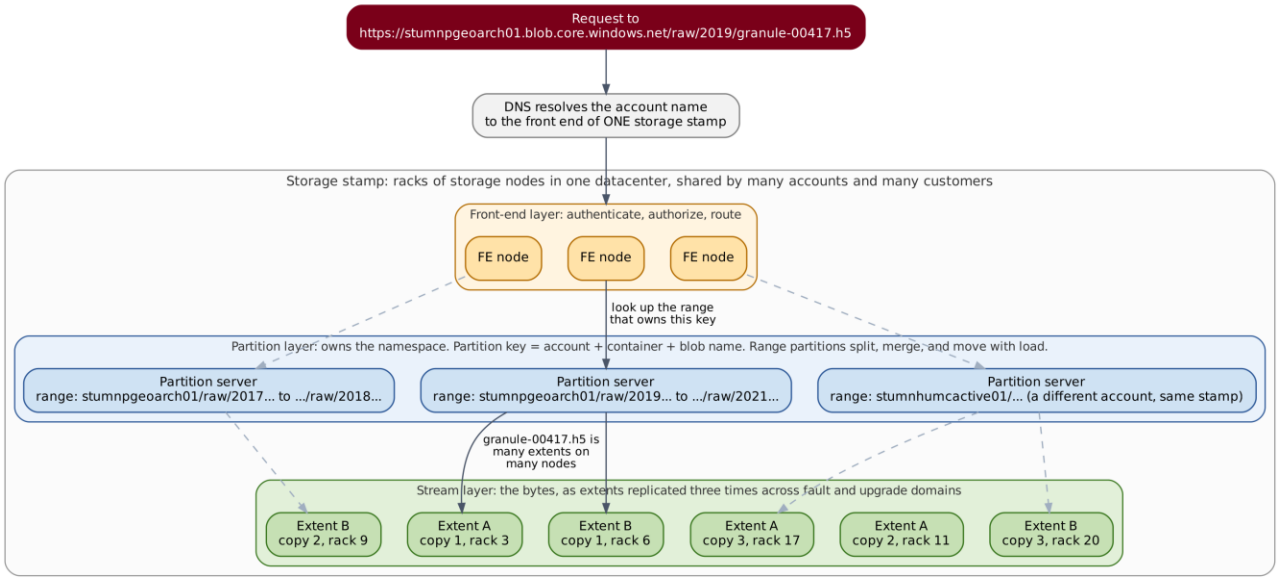


Figure 1. One request to a sharded account, traced through the stamp. The partition server that owns the name range can change without any data moving; the bytes live in the stream layer on many nodes.

- **Front-end layer.** Authenticates and authorizes the request, then looks up which partition server currently owns the key.
- **Partition layer.** Owns the namespace. Every blob has a partition key made of the account name, the container name, and the blob name. Keys are grouped into range partitions, each range partition is served by one partition server at a time, and the platform splits, merges, and moves range partitions across servers continuously as load shifts.
- **Stream layer.** Holds the bytes as extents (chunks of data). Each extent is replicated three times across separate fault domains and upgrade domains, meaning different racks and nodes. A large blob is many extents, so its bytes are already spread across many nodes. Moving a range partition to another partition server moves no data, because the data lives here.

6.3 Three consequences for sharding

1. **Two accounts in the same region may share a stamp.** Sharding buys independent quotas, independent policy, and independent blast radius. It does not buy hardware isolation. Only region, zone redundancy, and performance tier (premium accounts live on SSD-backed stamps) influence physical placement, and even those choose a stamp class, not a box.
2. **The hot-partition problem lives inside a shard.** Because partitioning is range based on the full blob name, lexically adjacent names land in the same range partition and on the same partition server until the platform rebalances. That is the source of the 503 Server Busy and 500 Operation Timeout responses, and it can fire while the account as a whole is far below its targets.
3. **A single blob has its own targets.** A single block blob is targeted at up to 3,000 requests per second. Sharding across accounts never helps a workload that hammers one object; redesigning the object (splitting it, caching it, fronting it with a CDN) does.

The analogy for the room

The stamp is the frame. The account is a volume with a QoS policy, a single redundancy setting, and its own keys. The partition server is the controller that happens to own that namespace range right now, and the controller can change without the data moving. The one thing the frame analogy gets wrong is placement: on the frame you chose the shelf; in Azure the platform does, and you cannot ask.

7. Naming is sharding one level down

Because the partition key is the full blob name, the naming convention inside an account is itself a sharding decision. The same idea that spreads a portfolio across accounts spreads a workload across range partitions within one.

Pattern	Example blob names	Effect on the partition layer	When it is fine
Sequential or timestamp-first	`telemetry/2026-10-20T04:55:01.bin` `telemetry/2026-10-20T04:55:02.bin`	Every new write lands at the end of the same range; one partition server takes the whole write rate until the platform splits the range	Low-rate archival writes, or data that must be listed by time range and is written in bulk once
Hash prefix	`a7f3/telemetry/2026-10-20T04:55:01.bin` `c21e/telemetry/2026-10-20T04:55:02.bin`	Adjacent writes land in different ranges and different partition servers from the first request	High-rate ingest, millions of small objects, anything that has thrown 503 responses
Reversed or salted ID	`714000-DS/granule.h5` `814000-DS/granule.h5`	Same effect as a hash prefix, with the original ID still recoverable from the name	Sequential IDs where you cannot change the ID itself but can change its position in the name
Instrument or unit prefix	`modis/2026/10/20/granule-00417.h5` `viirs/2026/10/20/granule-00417.h5`	Spreads load across as many ranges as there are prefixes; each prefix is still sequential inside	Natural for satellite and sensor data; combine with a hash when one instrument dominates

Two practical rules. First, choose the convention before the first petabyte lands, because renaming is a copy. Second, make the client retry with exponential backoff on 503 and 500 responses regardless of naming; the Azure SDKs do this by default and the behavior eases the load on the hot partition while the platform rebalances.

8. The decision flowchart

Run every dataset in the inventory through the questions below, in order. The first yes decides the outcome. The order matters: the two forced triggers come before the two judgment calls, and naming is fixed before any scale-driven split.

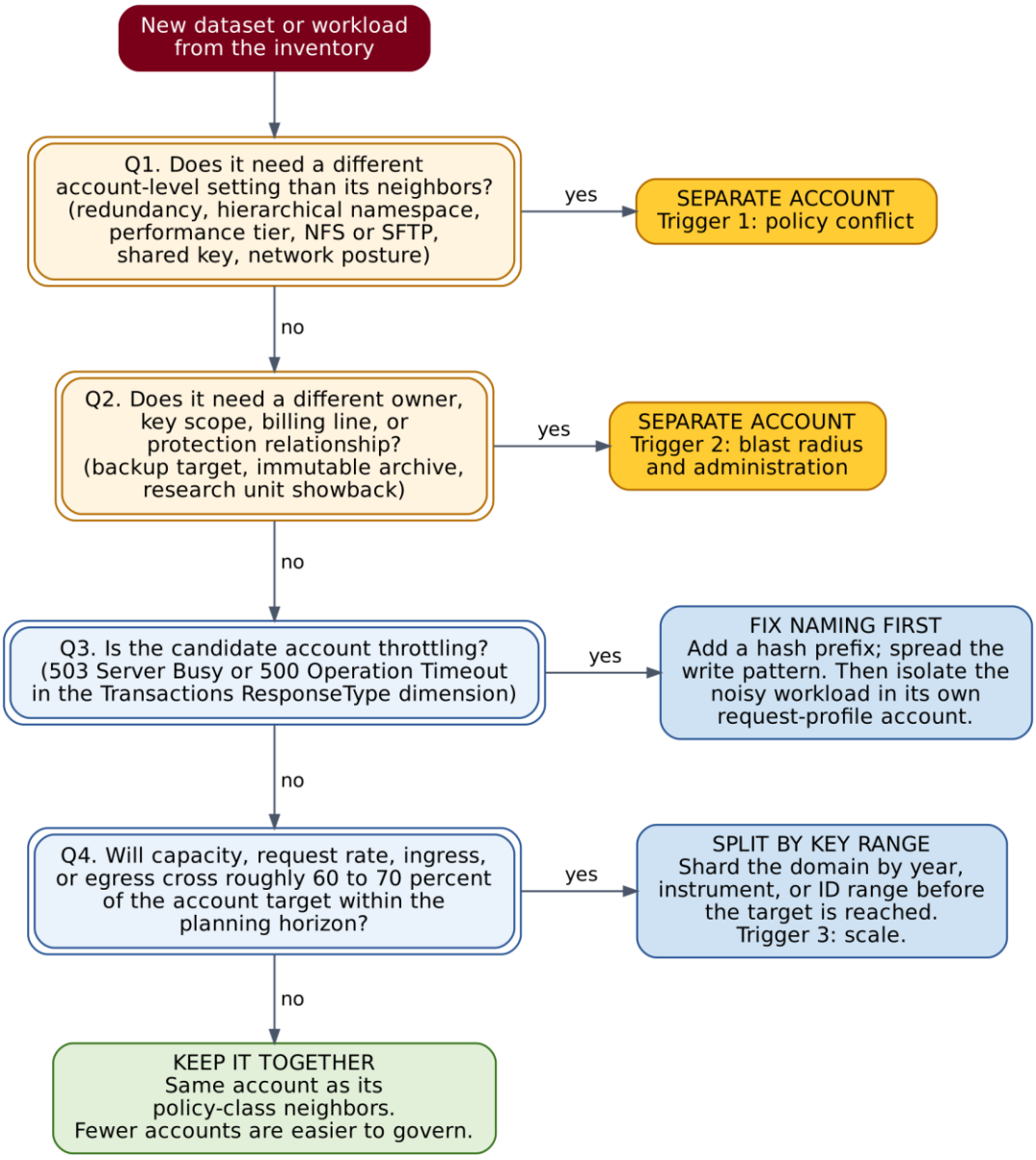


Figure 2. The sharding decision, in the order the triggers are checked. The first yes decides.

The ten-second rule

Policy conflict means separate accounts. Different owner or protection need means separate accounts. Throttling means fix naming, then isolate the noisy workload. Capacity or bandwidth trending toward a target means split the domain by key range before you reach it. Everything else stays together.

9. Worked example from the Research Storage Inventory

The UMN Research Storage Inventory workbook carries 12,240 synthetic datasets totaling about 3.0 PB, each with a research unit, data class, sensitivity, retention class, file count, average object size, capacity, 90-day access count, immutability flag, redundancy preference, and a sharded account name column. The nine datasets below are representative rows in the inventory's shape, chosen so that every trigger in this handout appears at least once. Figures are synthetic.

9.1 The naming convention

Storage account names must be 3 to 24 lowercase letters and digits and globally unique. The convention used by the inventory and the Session 6 Bicep module is `st` (storage) + `umn` + a four-letter research unit code + a class code + a two-digit sequence. For example, `stumnpgeoarch01` is the first archive account for the polar geospatial unit. The sequence number exists so that a scale-driven split produces a predictable second name.

Unit code	Research unit (synthetic)	Class codes in use	Meaning
pgeo	Polar geospatial program (satellite telemetry and derived products)	`arch` `active`	Archive-tier holdings; active processing inputs and outputs
humc	Humanities digital corpus program	`active` `arch`	Searchable corpus with researchers on it; published, frozen editions
geno	Genomics core	`lake`	Hierarchical namespace data lake for pipeline directories
hpcs	Research computing (HPC)	`scratch`	Premium scratch landing for cluster jobs
enta	Enterprise applications	`bkup` `files`	Immutable database backup target; SMB file shares

9.2 Nine representative datasets

Dataset	Unit	Data class	Capacity	Files and average object	90-day accesses	Immutable	Tier and redundancy	Account
DS-004117	pgeo	Satellite raw archive, 2009 to 2015	412 TB	18.2 M at 23 MB	3	Yes, 340 yr	Archive, GRS	`stumnpgeoarch01`
DS-004288	pgeo	Satellite raw archive, 2016 to 2021	388 TB	16.9 M at 23 MB	11	Yes, 340 yr	Archive, GRS	`stumnpgeoarch01`
DS-007302	pgeo	Active mosaic processing	61 TB	2.1 M at 29 MB	48,900	No	Hot, ZRS	`stumnpgeoactive01`
DS-001954	humc	Searchable text corpus	14 TB	41.0 M at 350 KB	1,240,000	No	Hot, ZRS	`stumnhumcactive01`
DS-002211	humc	Published corpus editions	9 TB	6.4 M at 1.4 MB	160	Yes, 100 yr	Cool then Archive, GRS	`stumnhumcarch01`
DS-009871	geno	Sequencing runs and pipeline outputs (Restricted)	290 TB	3.8 M at 76 MB	22,700	Yes, 25 yr	Hot then Cool, ZRS	`stumngenolake01`
DS-010443	hpcs	Cluster scratch landing	120 TB	9.6 M at 12 MB	3,900,000	No	Premium, ZRS	`stumnhpcsscratch01`

Dataset	Unit	Data class	Capacity	Files and average object	90-day accesses	Immutable	Tier and redundancy	Account
DS-011590	enta	Oracle RMAN backup pieces	85 TB	0.4 M at 212 MB	2,100	Yes, 7 yr	Cool, GZRS	`stumnentabkup01`
DS-003377	enta	Departmental SMB file shares	38 TB	27.5 M at 1.4 MB	890,000	No	Azure Files, ZRS	`stumnentafiles01`

9.3 Walking each dataset through the flowchart

Dataset	First yes	Outcome and reasoning
DS-004117 and DS-004288	Trigger 1 (policy)	Archive tier requires LRS, GRS, or RA-GRS, so these cannot share an account with anything zone redundant. Both are the same protection class and the same owner, so they share one account. Capacity of both together is 800 TB; the unit's full archive holdings are about 1.9 PB today, growing by roughly 280 TB a year, which crosses 70 percent of 5 PiB in about six years. The second shard is already named: stumnpgeoarch02, split by acquisition year.
DS-007302	Trigger 1 (policy)	Needs ZRS and Hot; cannot sit in the GRS archive account. Shares an account with the unit's other active ZRS data. Naming uses an instrument prefix, which is enough at 48,900 accesses in 90 days.
DS-001954	Trigger 1 (policy), then throttling	ZRS, so not the archive account. Separate from DS-007302 because the request profile is different: 41 million small objects and about 1.2 million accesses in 90 days, with peaks measured at 9,000 requests per second during indexing. That is within the 40,000 target for East US 2 but the corpus was named by sequential document ID, which clustered the indexing writes and produced 503 responses in the pilot. The fix was a four-character hash prefix; the account is also kept separate so that an indexing run cannot throttle the mosaic processing in stumnpgeoactive01.
DS-002211	Trigger 1 (policy)	Immutable with a 100-year retention and destined for Archive, so GRS and a flat namespace with version-level WORM. Different protection class from the active corpus, so its own account even though it is the same research unit.
DS-009871	Trigger 1 (policy)	The pipelines need directory rename and POSIX ACLs, so a hierarchical namespace account. That gives up versioning and version-level WORM, so the 25-year immutability requirement is met with a container-level time-based retention policy, which hierarchical namespace accounts do support. Restricted sensitivity also means a private endpoint only posture, which differs from the collaboration accounts.
DS-010443	Trigger 1 (policy)	Premium block blob for latency, which is a different account kind and performance tier. Premium block blob supports LRS and ZRS only and has no access tiers; a lifecycle delete rule clears scratch after seven days. The 3.9 million accesses in 90 days are isolated here by design, so a cluster job can never throttle an archive.
DS-011590	Trigger 2 (blast radius)	An immutable backup target. Separate account and separate subscription from the databases it protects, with a locked time-based policy and a different owning team. GZRS so that the backup survives a zone loss and a regional loss.
DS-003377	Trigger 1 (policy)	Azure Files SMB with identity-based access needs one identity source per account; the share lives in an account configured for on-premises Active Directory Domain Services, which nothing else in the portfolio needs.

9.4 The resulting account set

Account	Kind and tier	Namespace	Redundancy	Immutability	Network posture	Owner and billing
stumnpgeoarch01	Standard GPv2	Flat	GRS	Version-level WORM, locked	Private endpoint	Polar geospatial
stumnpgeoactive01	Standard GPv2	Flat	ZRS	None	Private endpoint plus allowed campus ranges	Polar geospatial
stumnhumcactive01	Standard GPv2	Flat	ZRS	None	Private endpoint plus allowed campus ranges	Humanities corpus
stumnhumcarch01	Standard GPv2	Flat	GRS	Version-level WORM, locked	Private endpoint	Humanities corpus
stumngenolake01	Standard GPv2	Hierarchical	ZRS	Container-level time-based policy	Private endpoint only	Genomics core
stumnhpcsscratch01	Premium block blob	Flat	ZRS	None; 7-day delete rule	Private endpoint only	Research computing
stumnentabkup01	Standard GPv2	Flat	GZRS	Version-level WORM, locked	Private endpoint only; separate subscription	Data protection team
stumnentafiles01	Standard GPv2 (Files)	Not applicable	ZRS	Share snapshots and backup	Private endpoint; AD DS identity source	Enterprise applications

Eight accounts for nine datasets looks like a lot until you notice that only one boundary (the pgeo archive's future second shard) exists because of size. Every other boundary is forced by a policy difference or an ownership difference, and would exist at a tenth of the capacity. That is the lesson to carry into the Block 4 design exercise: start with the policy and ownership cuts, then check whether any single resulting account is on a path to a target.

9.5 The capacity projection, as the inventory computes it

The Summary sheet of the inventory computes capacity by proposed tier and research unit. The projection for the one scale-driven boundary works like this, and the same arithmetic applies to any account:

Input	Value (synthetic)	Source
Current capacity in stumnpgeoarch01	1.9 PB (about 1.69 PiB)	Inventory, filtered by sharded account name
Annual growth	280 TB per year	Inventory, capacity by acquisition year
Target	5 PiB default (about 5.63 PB)	Microsoft Learn scalability targets page
Planning threshold	70 percent of target, about 3.9 PB	Practitioner rule
Years until threshold	(3.9 minus 1.9) divided by 0.28, about 7 years	Arithmetic
Decision	Name stumnpgeoarch02 now; begin writing acquisition year 2029 and later to it when the convention says so; no data moves	Flowchart, Trigger 3

Two things make this easy rather than painful. The split is by acquisition year, so the key is already in the blob name and no existing data moves. And the account is created by the same Bicep module with the same

parameters, so the second shard inherits the same lifecycle policy, immutability settings, network posture, and tags on the day it is created.

10. The techniques, as a menu, and two clarifications

Technique	What it is	What it buys	When to reach for it
By data domain	One account or set per research unit or project	Cleanest ownership, RBAC scope, and showback	The default cut; almost every portfolio starts here
By protection class	Separate accounts per combination of redundancy, namespace model, and immutability requirement	Satisfies Trigger 1; makes immutability and redundancy unambiguous per account	Whenever two datasets need different account-level answers
By request profile	Isolate transaction-heavy workloads (ingest, millions of small objects, scratch) from bulk data	A noisy workload cannot throttle a quiet one; premium tier where latency matters	When throttling appears or when the inventory shows a high file count and small average object size
By key range or hash	Split one domain across accounts by year, instrument, or dataset ID range	Pure horizontal scale within a domain	Only when one domain alone will exceed a target; the last technique you reach for
Within an account, by naming	Choose blob names so hot write patterns spread across range partitions	Avoids hot partitions and 503 responses without any new account	Always; it is the first fix for throttling

10.1 Containers are not shards

A container adds namespace, an RBAC scope, container-level immutability, and a prefix for lifecycle rules. It adds no quota, because every target in section 1.1 is per account. Splitting one account into fifty containers changes nothing about its capacity, request rate, or bandwidth limits. Containers organize; accounts shard.

10.2 Geo-redundancy is not sharding

GRS, GZRS, and their read-access variants copy the whole account to a fixed paired region chosen by Microsoft. That is replication: the same data in two places. Sharding is the customer deciding where cut lines go between different data. The two combine freely (each shard can have its own redundancy setting), but they answer different questions: redundancy answers "what survives a failure," sharding answers "what shares a quota and a policy."

11. Knowledge check

Answer these before the Block 4 design exercise. The key is on the last page.

1. A research unit has 600 TB of data in one account and wants to add a 20 TB dataset that must be zone redundant and a 40 TB dataset that will go to the Archive tier. The existing account is GRS. How many accounts are needed at minimum, and why?

2. An account is at 1.2 PB of capacity and 4,000 requests per second, well under every target, yet the Transactions metric shows a steady stream of ServerBusyError responses during a nightly ingest. What is the most likely cause and what is the first fix?
3. Two storage accounts are created five minutes apart in the same region with the same settings. Could their data be on the same physical racks? What, if anything, does creating the second account isolate?
4. A team proposes putting the Oracle RMAN backup target in a container inside the same account as the database VM's data disks "to keep things together." Which trigger does this violate and what is the recommended structure?
5. A single 180 GB blob is read by 400 compute nodes at once and the reads are slow. Would sharding the account help? What would?
6. The inventory projects that one archive account will reach 70 percent of 5 PiB in four years. What should happen now, what should happen then, and how much data moves?

12. Glossary

Term	Meaning
Storage account	The unit of quota, policy, keys, and billing in Azure Storage. A logical and administrative boundary, not a physical one.
Shard	One storage account in a set that together hold a portfolio, with a key and a map that say which account holds what.
Storage stamp	A cluster of racks of storage nodes in one datacenter, hosting many accounts from many customers. The physical unit.
Partition key	Account name plus container name plus blob name. Determines which range partition, and therefore which partition server, owns a blob's metadata and request handling.
Range partition	A contiguous lexical range of partition keys served by one partition server at a time. Split, merged, and moved by the platform as load shifts.
Hot partition	A range partition receiving more load than one partition server can serve. Produces 503 Server Busy and 500 Operation Timeout responses before the account reaches its documented limits.
Extent	A chunk of data in the stream layer, replicated three times across separate fault and upgrade domains. A large blob is many extents.
Fault domain and upgrade domain	Groupings of hardware that fail together (a rack, a power feed) or are updated together. Replicas are placed in different ones.
Scalability target	A published, achievable high-end figure for an account or object. Not a guarantee; depends on object size and access pattern.
Request profile	The transaction shape of a workload: object count, object size, request rate, read and write mix. Independent of capacity.
Protection class	The combination of redundancy, namespace model, and immutability requirement a dataset needs. Datasets in different classes cannot share an account.
Blast radius	Everything a key, a role assignment, a lifecycle rule, or a delete can reach. In Azure Storage, the account.

13. Microsoft Learn references

Verified October 2026. The first entry is the authority for every numeric target in this handout.

- Scalability and performance targets for standard storage accounts: `https://learn.microsoft.com/azure/storage/common/scalability-targets-standard-account`
- Scalability and performance targets for Blob storage (single-blob targets, partition key, hot partition guidance): `https://learn.microsoft.com/azure/storage/blobs/scalability-targets`
- Performance and scalability checklist for Blob storage (naming, backoff, concurrency): `https://learn.microsoft.com/azure/storage/blobs/storage-performance-checklist`
- Storage account overview (account types, naming rules, redundancy, hierarchical namespace): `https://learn.microsoft.com/azure/storage/common/storage-account-overview`
- Azure Storage redundancy: `https://learn.microsoft.com/azure/storage/common/storage-redundancy`
- Access tiers for blob data (Archive tier redundancy restriction): `https://learn.microsoft.com/azure/storage/blobs/access-tiers-overview`
- Blob storage feature support in Azure Storage accounts (what hierarchical namespace, NFS 3.0, and SFTP give up): `https://learn.microsoft.com/azure/storage/blobs/storage-feature-support-in-storage-accounts`
- Monitor Azure Blob Storage (metrics, ResponseType dimension): `https://learn.microsoft.com/azure/storage/blobs/monitor-blob-storage`
- Increase Azure Storage account quotas: `https://learn.microsoft.com/azure/quotas/storage-account-quota-requests`
- Background reading on the stamp, partition, and stream layers: Calder et al., "Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency," SOSP 2011 (Microsoft Research). The layer names in section 6 come from this paper; the architecture it describes remains the basis of the service.

Answer key

Question	Answer
1	Three. Archive tier requires LRS, GRS, or RA-GRS, so the archive dataset can join the existing GRS account. The zone-redundant dataset cannot, because redundancy is an account-level setting, so it needs a ZRS account. That is two accounts for data, and if the question's 600 TB and the archive dataset are different protection classes (say the archive needs locked immutability and the rest does not), the archive is better in a third. The minimum defensible answer is two; the recommended answer is three, and neither has anything to do with capacity.
2	A hot partition. The ingest writes names that are lexically adjacent (timestamps or sequential IDs), so every write lands on one partition server until the platform splits the range. First fix: change the naming to a hash or salted prefix so adjacent writes spread across ranges, and confirm the client retries with exponential backoff. Sharding the account would not help, because the same naming would recreate the same hot range in the new account.
3	Yes. Accounts in the same region are placed on stamps by the platform based on capacity and load, and two accounts can share a stamp, racks, and even partition servers. The second account isolates quota (its own 5 PiB, request rate, ingress, and egress), policy (its own redundancy, namespace, immutability, network posture), and blast radius (its own keys, RBAC, lifecycle, and delete scope). It does not isolate hardware.
4	Trigger 2, blast radius and administration. A container inside the same account shares the account's keys, RBAC, and deletion scope with the data it is supposed to protect; a compromised key or a mistaken account delete takes both. Recommended structure: a separate storage account in a separate subscription owned by the data protection team, with a locked time-based immutability policy, as in the Session 3 and Session 5 designs.
5	No. A single block blob has its own target of up to 3,000 requests per second, and that target is per object regardless of which account holds it. What helps: split the object into parts read in parallel, raise the request size so fewer requests move more data, cache it on the compute side or behind a CDN, or stage a copy per group of nodes. Throughput of a single blob can approach the account limits when requests are large and concurrent, so request size matters more than account count.
6	Now: name the second shard according to the convention (sequence 02), define the key that will route data to it (acquisition year, instrument, or ID range), and make sure the Bicep module and the inventory's account-name formula know about it. Then: create the account and start writing new data that matches the key to it. Data moved: none. The split is by key range on new writes, so existing blobs stay where they are and nothing is copied.

End of handout. Bring your flowchart outcomes for the three Block 4 data classes (polar satellite archive, humanities corpus with active researchers, enterprise application data) to the design exercise.